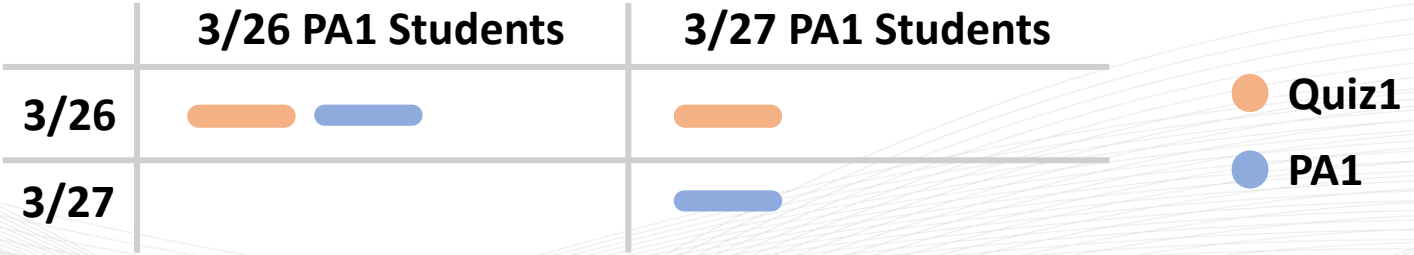
Computer Organization 2026
Quiz 1 & Programming Assessment I

Announce date: 2026.3.19

Assessment date: 2026.3.26 & 2026.3.27

Quiz1 Overview

- Date: 3/26
- Time: 09:00 - 09:30
- Rooms: A1302 (For 3/27 PA1 Students), 65203, 65304
- Regardless of whether your PA1 is on 3/26 or 3/27, **all must attend Quiz1 on 3/26**.



PA1 Overview

- Date: 3/26, 3/27
- Time: 09:30 - 13:00
- Rooms: 65203, 65304
- You are asked to write **inline assembly code** and run it on RISC-V Spike simulator to get results.
- The goal is to practice low-level programming and understand how assembly interacts with C.

Prerequisite: HW0 – Environment Setup

- You have to follow the procedures listed in the **HW0** document to set up your development environment.
- We will use **Docker** for 3/26 & 3/27 programming assessment. Make sure you're familiar with basic Docker operations.

Assessment - 3 Coding Problems

- Problem 1: Insertion Sort, 30%
- Problem 2: Binary Search, 30%
- Problem 3: Linked-List Cycle, 40%

Problem 1. Insertion Sort (30%)

- Input:
  - The first line contains an integer, the **size of the array**.
  - The second line contains floating point **key values**.
- Output:
  - The first line contains sorted array.
  - The second line contains an integer, shift count, which is the times of elements shifted.

Input:

```
5
5.0 2.0 4.0 6.0 1.0
```

Expected output:

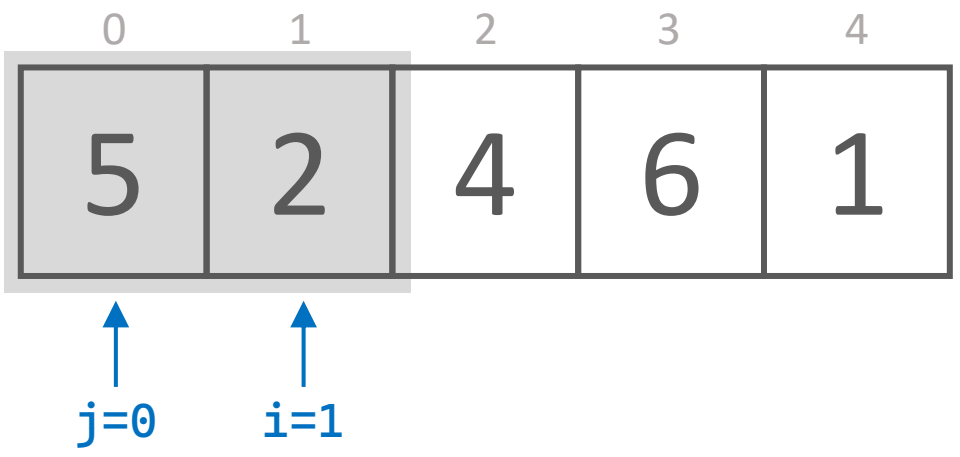
```
1.0 2.0 4.0 5.0 6.0
6
```

Problem 1. Insertion Sort (Cont.)

Algorithm 1 Insertion Sort with Shift Count

```
procedure INSERTION-SORT(A)
    n ← length(A)
    shift_cnt ← 0
    for i ← 1 to n − 1 do
        key ← A[i]
        j ← i − 1
        while j ≥ 0 and A[j] > key do
            A[j + 1] ← A[j]
            j ← j − 1
            shift_cnt ← shift_cnt + 1
        end while
        A[j + 1] ← key
    end for
    return shift_cnt
end procedure
```

Main array



Inputs

array A: [5, 2, 4, 6, 1]

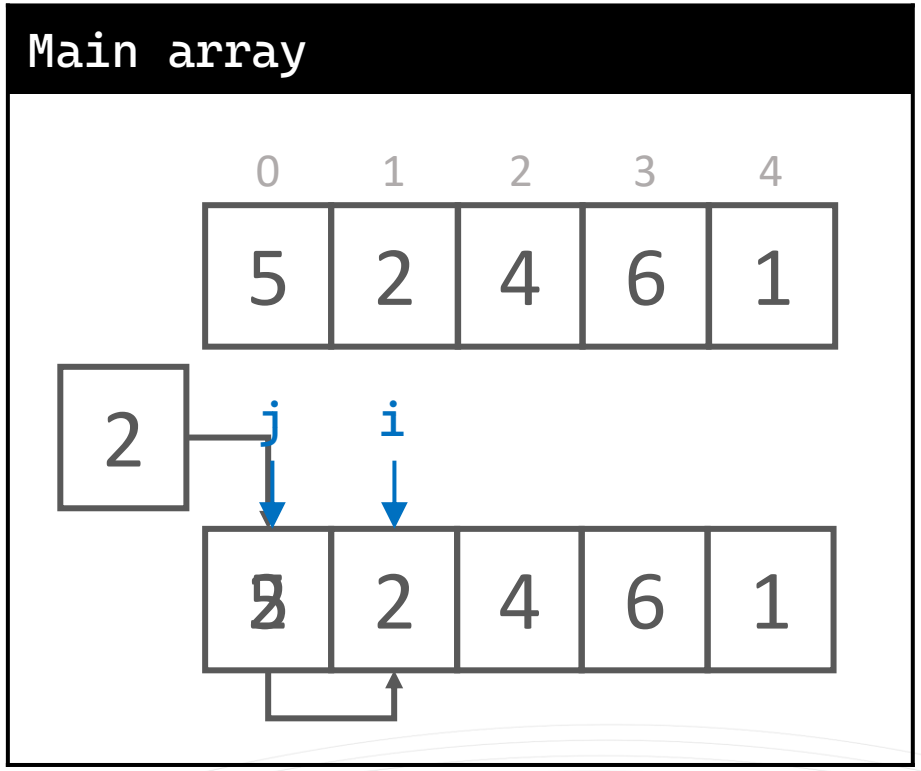
n: 5

shift_cnt: 0

Problem 1. Insertion Sort (Cont.)

1st iteration: Insert 2 (i = 1)

1. key = A[1] = 2
2. Compare: A[0] = 5 & key = 2 (5 > 2)
3. Shift: 5 shift to A[1]
4. shift_cnt + 1 = 0 + 1 = 1
5. Insert: 2 insert to A[0]



State

Current array: [2, 5, 4, 6, 1]

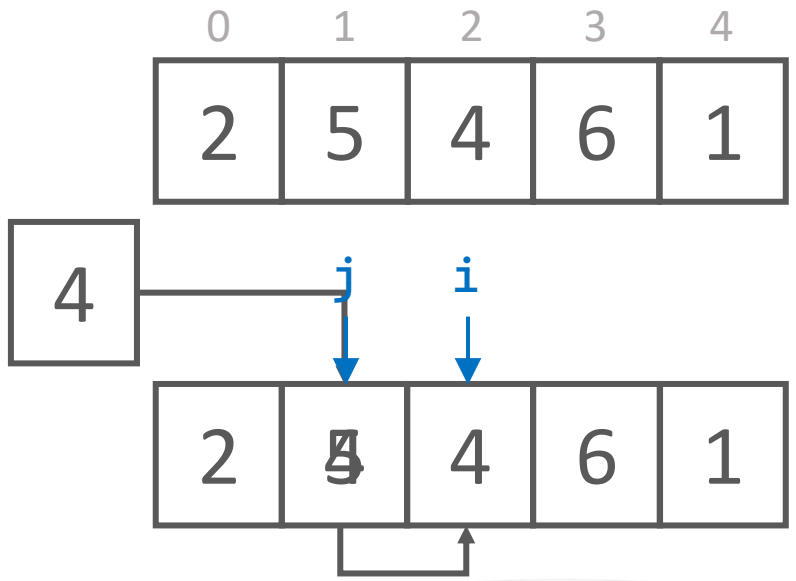
shift_cnt: 1

Problem 1. Insertion Sort (Cont.)

2nd iteration: Insert 4 (i = 2)

1. key = A[2] = 4
2. Compare: 5 > 4 → Shift 5 (shift_cnt + 1)
3. Compare: 2 > 4 → False (stop)
4. Insert: 4 insert to A[1]

Main array



State

Current array: [2, 4, 5, 6, 1]

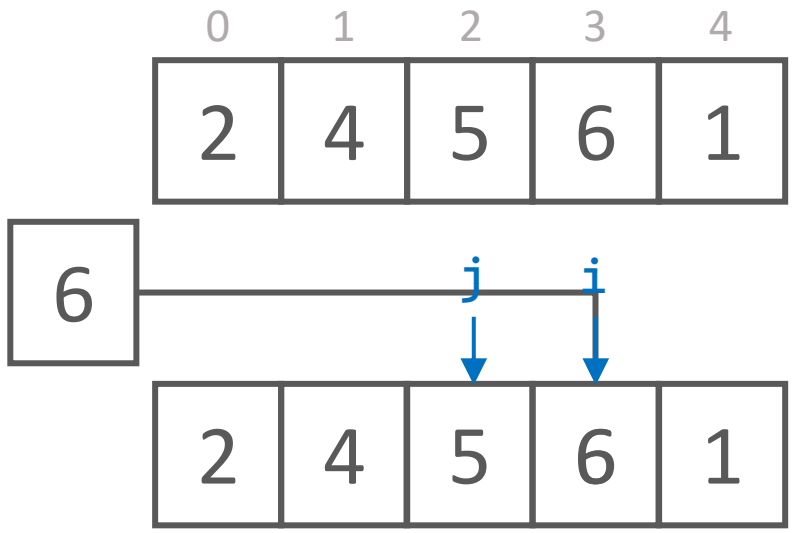
shift_cnt: 2

Problem 1. Insertion Sort (Cont.)

3rd iteration: Insert 6 (i = 3)

1. key = A[3] = 6
2. Compare: 5 > 6 → False (stop)
3. No shift
4. Insert: 6 insert to A[3]

Main array



State

Current array: [2, 4, 5, 6, 1]

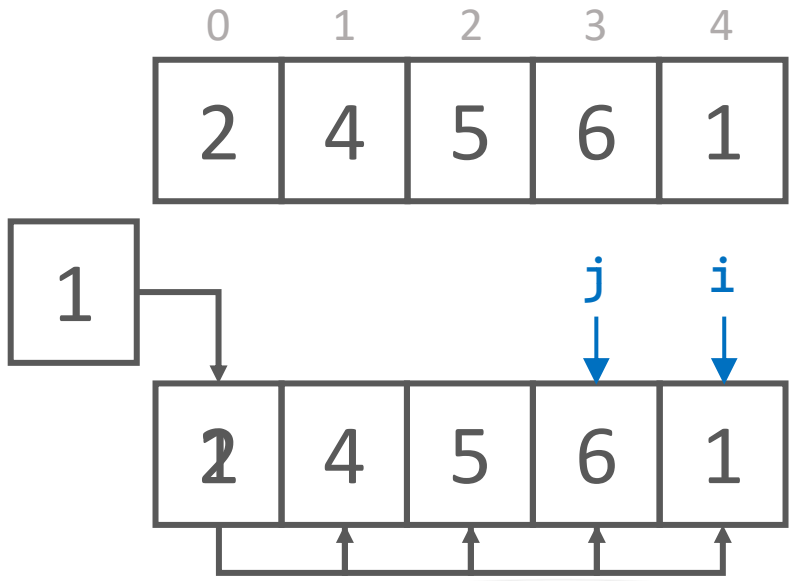
shift_cnt: 2

Problem 1. Insertion Sort (Cont.)

4th iteration: Insert 1 (i = 4)

1. key = A[4] = 1
2. Compare: 6 > 1 → Shift 6 (shift_cnt + 1)
3. Compare: 5 > 1 → Shift 5 (shift_cnt + 1)
4. Compare: 4 > 1 → Shift 4 (shift_cnt + 1)
5. Compare: 2 > 1 → Shift 2 (shift_cnt + 1)
6. Insert: 1 insert to A[0]
7. Finish

Main array



Result

Sorted array: [1, 2, 4, 5, 6]

shift_cnt: 6

Problem 2. Binary Search (30%)

- Input:
  - The first line contains an integer, target, to search for in the array.
  - The second line contains the array size filled by Problem 1.
  - The third line contains the sorted array filled by Problem 1.
- Output:
  - A single integer representing the index of the target in the array, or -1 if the target is not found.

Input:

```
4
? # 5
? # 1.0 2.0 4.0 5.0 6.0
```

Expected output:

```
2
```

Problem 2. Binary Search (Cont.)

Algorithm 2 Binary Search

```
1:  procedure BINARY-SEARCH(A, target)
2:      left ← 0
3:      right ← length(A) − 1
4:      result ← −1
5:      while left ≤ right do
6:          mid ← ⌊(left + right)/2⌋
7:          if A[mid] = target then
8:              result ← mid
9:              break
10:         else if A[mid] < target then
11:             left ← mid + 1
12:         else
13:             right ← mid − 1
14:         end if
15:     end while
16:     return result
17: end procedure
```

Inputs

```
target: 4
Array size: 5
Sorted array: [1, 2, 4, 5, 6]


# Note: The array size (line 2) and
  the sorted array (line 3) will be
  filled by Problem 1.
```

Result

```
Index: 2
```

Problem 3. Linked-List Cycle (40%)

- Input:
  - The first line contains two integers, the **number of nodes** and the **number of edges**.
  - The second line contains **key values** of the input nodes.
  - Each of the lines contains two integers `from` and `to`, representing a directed edge from `from` to `to` (e.g., 1 2 means a directed edge from 1 to 2).
- Output:
  - The first line contains an integer **key value of meet pointer**.
  - The second line contains the result of having cycle or not. (e.g., Has cycle: true; no cycle: false)

Input:

```
3 3
2 1 3
2 1
1 3
3 2
```

Expected output:

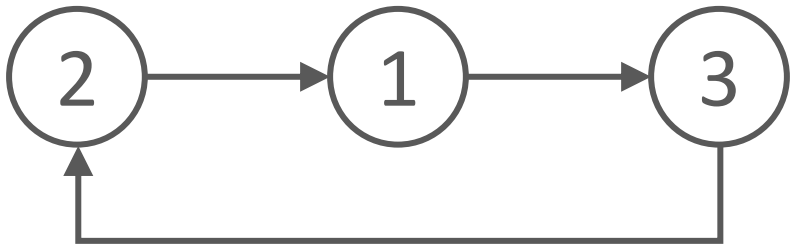
```
2
true
```

Problem 3. Linked-List Cycle (Cont.)

The Tortoise and Hare Algorithm

- The tortoise moves one step at a time, and the hare moves two steps at a time.
- Given a path, we want to determine whether the path contains a cycle.
- If so, at which position do the hare and the tortoise meet for the first time inside the cycle?

Main list



Inputs

```
Node count: 3
Edge count: 3
Node: 2, 1, 3
Edge: (2,1), (1,3), (3,2)
```

Problem 3. Linked-List Cycle (Cont.)

The Tortoise and Hare Algorithm

- T1:
  - The hare moves two steps.
  - The tortoise moves one step.

→ Not meet yet.

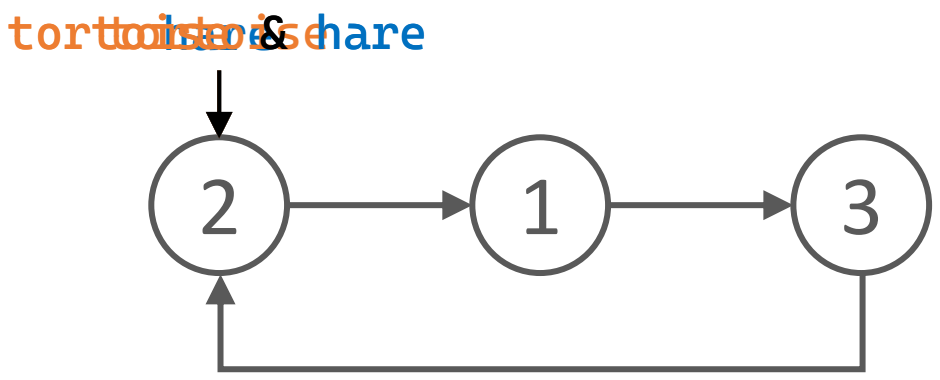
- T2:
  - The hare moves two steps.
  - The tortoise moves one step.

→ Not meet yet.

- T3:
  - The hare moves two steps.
  - The tortoise moves one step.

→ Meet!

Main list



Result

key: 2

Contains cycle: true

Test Your Assessment

- Please use **local-judge** according to the **HW0** instructions to test your program.
- You have two ways to run tests:
  - Use `make judge` to test all problems.
  - Use `judge -c judge.conf` to test a specific problem.

`make judge`

- Test all exercises and all public testcases.

```
Ubuntu
root@a5eeae30f02e:/workspace/CO_StudentID_PE1# make judge
riscv64-unknown-linux-gnu-gcc -static -o 1 1_insertion_sort.c
riscv64-unknown-linux-gnu-gcc -static -o 2 2_binary_search.c
riscv64-unknown-linux-gnu-gcc -static -o 3 3_linked_list_cycle.c
judge -c judge1.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   1_1 | ✔
=======+==============================================================================
   1_2 | ✔
=======+==============================================================================
Correct/Total problems: 2/2
Obtained/Total scores:  20/20
or `judge -c judge1.conf -v 1`
judge -c judge2.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   2_1 | ✔
=======+==============================================================================
   2_2 | ✔
=======+==============================================================================
Correct/Total problems: 2/2
Obtained/Total scores:  20/20
or `judge -c judge2.conf -v 1`
judge -c judge3.conf
local-judge: v2.7.4
=======+==============================================================================
Sample | Accept
=======+==============================================================================
   3_1 | ✔
=======+==============================================================================
   3_2 | ✔
=======+==============================================================================
Correct/Total problems: 2/2
Obtained/Total scores:  20/20
or `judge -c judge3.conf -v 1`
```

`judge -c judge$i.conf -v 1`

- Test a specific problem.
- Add `-v 1` to the command if you need more information.

```
Ubuntu
root@a5eeae30f02e:/workspace/CO_StudentID_PE1# judge -c judge1.conf -v 1
local-judge: v2.7.4
=======+==================================================================================
Sample | Accept
=======+==================================================================================
   1_1 | ✗
-------+----------------------------------------------------------------------------------
--- /workspace/testcases/expected/1_1.out       2026-02-10 10:03:16.834147073 +0000
+++ ../output/1_1local_1771082321.out   2026-02-14 15:18:41.784763619 +0000
@@ -1,2 +1,2 @@
-1 2 3 4 5 6 7 8 9 10
-27
+9 5 8 8 8 8 8 8 8 8
+1

=======+==================================================================================
   1_2 | ✗
-------+----------------------------------------------------------------------------------
--- /workspace/testcases/expected/1_2.out       2026-02-10 10:03:16.834147073 +0000
+++ ../output/1_2local_1771082321.out   2026-02-14 15:18:41.804763619 +0000
@@ -1,2 +1,2 @@
-1 3 5 6 8 13 15 17 18 19 21 22 23 26 27 28 31 34 35 36 37 38 39 40 42 43 44 45 48 51
-626
+23 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65 65
+0

=======+==================================================================================
Correct/Total problems: 0/2
Obtained/Total scores:  0/20
```

Scoring Criteria

- Total score: 100 points
- Problem 1: Insertion Sort, 30%
  - 2 public testcases (20%)
  - **2 hidden testcases (10%)**
- Problem 2: Binary Search, 30%
  - 2 public testcases (20%)
  - **2 hidden testcases (10%)**
- Problem 3: The Tortoise and the Hare, 40%
  - 2 public testcases (20%)
  - **4 hidden testcases (20%)**

Important Notes

- The computers in the exam environment will use **Ubuntu**. We will provide **VSCode** for students to use, the **container** will be pre-loaded, and no additional extensions will be installed.
- During the computer-based exam:
  - No entry is allowed **15 minutes** after the Quiz1 and PA1 begins.
  - The network need to be completely disconnected.
  - You may only access the NCKU Moodle while uploading your submission.
  - The use of USB/external devices, cheat sheets, paper notes, or any AI tools is strictly prohibited.
  - Violations of the above rules will be regarded as cheating, and the exam score will be recorded as **0**.
  - There will be no hint provided during the exam.

Contact Information

- Please email asrlab@mail.csie.ncku.edu.tw with any assessment issue or grading question before **12:00PM on March 24, 2026**, email subject starts with [Comp2026].
- Office Hours: Wednesdays 11:30–13:00, Room 65704, ASRLab.

3/19 09:00
- **Problem release**

3/24 12:00
- **Email reply deadline**

3/26 & 3/27 09:00
- **09:00 – 09:30 Quiz1 &**
  09:30 – 13:00 PA1

4/23 23:59
- **Post grades**